

AI AND ML LABORATORY

LAB MANUAL

IFETCE R-2023



IFET COLLEGE OF ENGINEERING

[An Autonomous Institution]

DEPARTMENT OF CSE (Cyber Security)

SUBJECT CODE: 23CY5L01

YEAR/SEM: III/V

<u>Prepared by:</u>	<u>Verified by:</u>	<u>Approved by:</u>
Mrs.R.Senthamarai, AP/CSE(CS)	Mrs.J.Jayachitra, HOD/CSE(CS).	Mrs.J.Jayachitra, HOD/CSE(CS).

DEPARTMENT VISION

To transform individuals into cyber security technocrats for creating Secure digital world.

DEPARTMENT MISSION

To maintain and improve our facilities to support innovative teaching, research and hands-on learning experiences.

To enhance the skill sets of the faculty through continuous improvement programs.

To inspire, educate and empower our students into globally competent professionals in cyber security field.

To promote seamless interaction between the department and the industrial community for internships, joint projects and knowledge exchange programs.

To instill and uphold the highest ethical standards by faculty and students in all aspects of life, education and professional practice.

PROGRAMME OUTCOMES (PO)

On the completion of the program the graduates are expected to know or develop the abilities defined through the Program Outcomes.

The Program Outcomes of Computer Science and Engineering (CS) Programme is given below.

- PO 1: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- PO 2: Identify, formulate, research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO 3: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO 4: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO 5: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
- PO 6: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO 7: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for

sustainable development.

PO 8: Ability to effectively integrate IT-based solutions into the user environment.

PO 9: Understanding of ethical, legal, and social issues and responsibilities relevant to professional computing practice.

PO 10: Broad education necessary to understand the impact of Information Technology and Engineering solutions in the societal and human context.

PO 11: Ability to assist in the creation of an effective project plan by understanding best practices and standards and their applications.

PO 12: Recognition of the need for, and have the ability to engage in continuous professional development and life-long learning.

PROGRAMME SPECIFIC OUTCOMES (PSO)

The Computer Science and Engineering (Cyber Security) Graduates will:

PSO 1: Apply fundamental principles and problem-solving skills to develop and automate business solutions.

PSO 2: Analyze security principles to design effective, innovative and ethically acceptable security solutions

PSO 3: Use research skills and the security tools for analyzing, designing, and implementing security systems for individuals, organizations, and society.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

PEO 1: Graduates of the programme will have successful professional career growth, ranging from information and network security analysts to managers in the field of cyber security.

PEO 2: Graduates of the programme will have the technical competency to develop security-based solutions for complex real-world and multidisciplinary problems in an economical way.

PEO 3: Graduates of the programme will continue to learn and adopt new cutting-edge technologies and evolve as responsible citizens with good leadership qualities and strong ethical values in their chosen field.

SYLLABUS:

23CY5L01

AI AND ML LABORATORY

L	T	P	C
0	0	3	1. 5

LEARNING OBJECTIVES:

The faculty will enhance the skills of the students to

- Familiarize the uniformed and informed search algorithm.
- Impart the knowledge of regression model and decision tree algorithm.
- Study the usage of SVM and k-nearest neighbour algorithm.
- Understand k-means and random forest algorithm.
- Learn how to model the Bayesian networks and ANN.

LIST OF EXPERIMENTS:

1. Implementation of Uninformed search algorithms (BFS, DFS).
2. Implementation of Informed search algorithms (A*, memory-bounded A*).
3. Build linear and non-linear Regression models.
4. Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.
5. Detect Spam mails using Support vector machine.
6. Implement k-nearest neighbours classification.
7. Implement k-means clustering algorithm.
8. Implement sentiment analysis using random forest optimization algorithm.
9. Write a program to construct a Bayesian network considering medical data.
10. Implement facial recognition application with artificial neural network.

TOTAL: 45 PERIODS

COURSE OUTCOMES:

Upon completion of the course, the students will be able to

CO1: Apply uninformed and informed search algorithms

CO2: Implement regression models and decision tree algorithm

CO3: Apply support vector machine and k-nearest neighbor algorithm to real world problems

CO4: Implement K-means clustering and random forest algorithm

CO5: Implement models such as Bayesian networks and artificial neural networks

CO – PO MAPPING:

	P	P	P	P	P	P	P	P	P	P	P	P	P	P	P
P	0	0	0	0	0	0	0	0	0	0	0	0	S	S	S
O	1	2	3	4	5	6	7	8	9	1	1	1	0	0	0
s															
C															
O															
s															
CO1	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2
CO2	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2
CO3	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2
CO4	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2
CO5	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2
CO	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2
Correlation levels:				1: Slight (Low)				2: Moderate (Medium)				3: Substantial (High)			

INDEX
TABLE OF CONTENTS

EX.NO	EXPERIMENT NAME	PAGE NO
1	Implementation of Uninformed search algorithms (BFS, DFS).	7
2	Implementation of Informed search algorithms (A*, memory-bounded A*).	10
3	Build linear and non-linear Regression models.	16
4	Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.	19
5	Detect Spam mails using Support vector machine.	22
6	Implement k-nearest neighbours classification.	26
7	Implement k-means clustering algorithm.	29
8	Implement sentiment analysis using random forest optimization algorithm.	31
9	Write a program to construct a Bayesian network considering medical data.	34
10	Implement facial recognition application with artificial neural network.	36

Date :

AIM

To implement Uninformed Search Algorithms – Depth-First Search (DFS) and Breadth-First Search (BFS) – to search for an element in a Binary Search Tree (BST) using Python.

ALGORITHM

1. Create a Binary Search Tree (BST) using a predefined list of elements.
2. Define a Node class and an insert() function to build the BST.
3. For DFS, use a stack to traverse the BST and search for the element.
4. For BFS, use a queue to traverse the BST and search for the element.
5. Return and display whether the element is found or not in each method.

PROGRAM

DFS in Binary Search Tree

class Node:

```
def __init__(self, key):  
    self.left = None  
    self.right = None  
    self.val = key
```

def insert(root, key):

```
    if root is None:  
        return Node(key)  
    if key < root.val:  
        root.left = insert(root.left, key)  
    else:  
        root.right = insert(root.right, key)  
    return root
```

def dfs_search(root, key):

```
    stack = [root]  
    while stack:  
        node = stack.pop()  
        if node:  
            if node.val == key:  
                return True  
            # Push right first, then left (so left is processed first)  
            stack.append(node.right)  
            stack.append(node.left)  
    return False
```

Main Program

```

lucky_numbers = [17, 3, 25, 1, 7, 20, 30]
root = None
for num in lucky_numbers:
    root = insert(root, num)

search_key = int(input("Enter the element to search using DFS: "))
result = dfs_search(root, search_key)

if result:
    print("Element found in BST using DFS.")
else:
    print("Element not found in BST using DFS.")

```

OUTPUT

```

Enter the element to search using DFS: 20
Element found in BST using DFS.

```

```

# BFS in Binary Search Tree
from collections import deque

```

```

class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.val:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

```

```

def bfs_search(root, key):
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node:
            if node.val == key:
                return True
            queue.append(node.left)
            queue.append(node.right)
    return False

```

```

# Main Program

```

```

lucky_numbers = [17, 3, 25, 1, 7, 20, 30]
root = None
for num in lucky_numbers:
    root = insert(root, num)

search_key = int(input("Enter the element to search using BFS: "))
result = bfs_search(root, search_key)

if result:
    print("Element found in BST using BFS.")
else:
    print("Element not found in BST using BFS.")

```

Output:
Enter the element to search using BFS: 7
Element found in BST using BFS.

RESULT

The Python programs for searching an element in a Binary Search Tree (BST) using Depth-First Search (DFS) and Breadth-First Search (BFS) were executed successfully and the output has been verified. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PS0	PS0	PS0
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO 1	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2

AIM

To implement informed search algorithms A* and memory-bounded A*

ALGORITHM

1. Start by reading the initial and goal states, and calculate the heuristic (Manhattan distance) from the initial state to the goal to guide the search.
2. For A*: Use a priority queue (open list) to store nodes, always expand the node with the lowest $f(n) = g(n) + h(n)$ value; For IDA*: Set an initial threshold equal to the heuristic of the start node and perform depth-first search bounded by this threshold.
3. At each step, check if the current node matches the goal; if yes, return the path from the start to this goal node.
4. Generate all possible neighboring states by moving the blank tile, and calculate their $f(n)$ values;
 - For A*: Add neighbors to the open list if they offer a better path.
 - For IDA*: Continue recursion only if $f(n)$ is within the threshold.
5. Repeat the process:
 - A*: Until the open list is empty or the goal is found.
 - IDA*: If the goal is not found in the current threshold, increase it to the smallest $f(n)$ that exceeded the limit and repeat the search.

PROGRAM

```
import heapq
```

```
# Heuristic: Manhattan distance
```

```
def manhattan_distance(state, goal):
```

```
    distance = 0
```

```
    for val in range(1, 9): # tiles 1-8, skip 0 (blank)
```

```
        idx1 = state.index(val)
```

```
        idx2 = goal.index(val)
```

```
        x1, y1 = idx1 // 3, idx1 % 3
```

```
        x2, y2 = idx2 // 3, idx2 % 3
```

```
        distance += abs(x1 - x2) + abs(y1 - y2)
```

```
    return distance
```

```
# Get neighbors by moving the blank tile (0)
```

```
def get_neighbors(state):
```

```
    neighbors = []
```

```
    idx = state.index(0)
```

```
    x, y = idx % 3, idx // 3
```

```
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)] # left, right, up, down
```

```
    for dx, dy in directions:
```

```
        nx, ny = x + dx, y + dy
```

```
        if 0 <= nx < 3 and 0 <= ny < 3:
```

```
            new_idx = ny * 3 + nx
```

```
            new_state = state[:] # copy state
```

```
            new_state[idx], new_state[new_idx] = new_state[new_idx], new_state[idx]
```

```
            neighbors.append(new_state)
```

```
    return neighbors
```

```
# Reconstruct solution path
```

```
def reconstruct_path(came_from, current):
```

```
    path = [current]
```

```
    while tuple(current) in came_from:
```

```
        current = came_from[tuple(current)]
```

```
        path.insert(0, current)
```

```
    return path
```

```
# A* Search
```

```
def a_star(start, goal):
```

```
    open_set = []
```

```
    heapq.heappush(open_set, (manhattan_distance(start, goal), 0, start))
```

```
    came_from = {}
```

```
    g_score = {tuple(start): 0}
```

```
    while open_set:
```

```
_, cost, current = heapq.heappop(open_set)
```

```
if current == goal:
```

```
    return reconstruct_path(came_from, current)
```

```
for neighbor in get_neighbors(current):
```

```
    tentative_g = g_score[tuple(current)] + 1
```

```
    if tuple(neighbor) not in g_score or tentative_g < g_score[tuple(neighbor)]:
```

```
        g_score[tuple(neighbor)] = tentative_g
```

```
        f = tentative_g + manhattan_distance(neighbor, goal)
```

```
        heapq.heappush(open_set, (f, tentative_g, neighbor))
```

```
        came_from[tuple(neighbor)] = current
```

```
return None
```

```
# IDA* Search
```

```
def ida_star(start, goal):
```

```
    def dfs(path, g, threshold):
```

```
        node = path[-1]
```

```
        f = g + manhattan_distance(node, goal)
```

```
        if f > threshold:
```

```
            return f
```

```
        if node == goal:
```

```
            return path
```

```
        min_threshold = float('inf')
```

```
        for neighbor in get_neighbors(node):
```

```
            if neighbor not in path: # avoid cycles
```

```
                path.append(neighbor)
```

```
                result = dfs(path, g + 1, threshold)
```

```
                if isinstance(result, list): # found solution
```

```
                    return result
```

```
                if result < min_threshold:
```

```
                    min_threshold = result
```

```
            path.pop()
```

```
return min_threshold
```

```
threshold = manhattan_distance(start, goal)
```

```
path = [start]
```

```
while True:
```

```
    result = dfs(path, 0, threshold)
```

```
    if isinstance(result, list):
```

```
        return result
```

```
    if result == float('inf'):
```

```
        return None
```

```
    threshold = result
```

```
# Input
```

```
def get_input():
```

```
    print("Enter the initial state (9 numbers, use 0 for blank):")
```

```
    start = list(map(int, input().split()))
```

```
    print("Enter the goal state (9 numbers):")
```

```
    goal = list(map(int, input().split()))
```

```
    return start, goal
```

```
# Output
```

```
def print_solution(solution):
```

```
    if solution is None:
```

```
        print("No solution found.")
```

```
    else:
```

```
        print("Steps to reach goal:")
```

```
        for state in solution:
```

```
            for i in range(0, 9, 3):
```

```
                print(state[i:i+3])
```

```
            print("——")
```

```
        print("Total steps:", len(solution) - 1)
```

```

# Main
if __name__ == "__main__":
    start, goal = get_input()

    print("\n— A* Search —")
    a_star_result = a_star(start, goal)
    print_solution(a_star_result)

    print("\n— Memory-Bounded A* (IDA*) —")
    ida_star_result = ida_star(start, goal)
    print_solution(ida_star_result)

```

OUTPUT

Start: 1 2 3 4 5 6 0 7 8

Goal: 1 2 3 4 5 6 7 8 0

— A* Search —

Steps to reach goal:

[1, 2, 3]

[4, 0, 5]

[6, 7, 8]

—

[1, 2, 3]

[4, 5, 0]

[6, 7, 8]

—

Total steps: 1

— Memory-Bounded A* (IDA*) —

Steps to reach goal:

[1, 2, 3]

[4, 0, 5]

[6, 7, 8]

—

[1, 2, 3]

[4, 5, 0]

[6, 7, 8]

—

Total steps: 1

RESULT

Thus, the python program to implement Informed search algorithms (A*, memorybounded A*) are executed successfully and the output is verified. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PS0	PS0	PS0
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO 1	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2

Date:

AIM

To Build linear and non-linear Regression models.

ALGORITHM

1. Collect, clean, and format the dataset into features (X) and target (y).
2. Fit a linear regression model to the cleaned data.
3. Transform features for polynomial regression (non-linear regression) and
4. fit the model.
5. Predict and compute performance metrics (Mean Squared Error and R²
6. score) for both models.
7. Plot predictions of linear regression and polynomial regression, then
8. compare visually and statistically.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# 1. Generate synthetic dataset
np.random.seed(42)
X = np.random.rand(100, 1) * 10 # Features from 0 to 10
y = 2.5 * X**2 + 1.5 * X + 5 + np.random.randn(100, 1) * 8 # Quadratic relationship with
noise

# 2. Linear Regression
lin_reg = LinearRegression()
lin_reg.fit(X, y)
y_pred_linear = lin_reg.predict(X)

# 3. Polynomial (Non-linear) Regression
degree = 2
poly_features = PolynomialFeatures(degree=degree)
X_poly = poly_features.fit_transform(X)
poly_reg = LinearRegression()
poly_reg.fit(X_poly, y)
```

```
y_pred_poly = poly_reg.predict(X_poly)
```

```
# 4. Metrics
```

```
print("Linear Regression:")
```

```
print(f" MSE: {mean_squared_error(y, y_pred_linear):.2f}")
```

```
print(f" R2 : {r2_score(y, y_pred_linear):.4f}")
```

```
print("\nPolynomial Regression (degree = 2):")
```

```
print(f" MSE: {mean_squared_error(y, y_pred_poly):.2f}")
```

```
print(f" R2 : {r2_score(y, y_pred_poly):.4f}")
```

```
# 5. Plot results
```

```
plt.figure(figsize=(10, 5))
```

```
# Scatter plot of data
```

```
plt.scatter(X, y, color="blue", label="Data Points", alpha=0.6)
```

```
# Sort X for smooth curve plotting
```

```
X_range = np.linspace(0, 10, 200).reshape(-1, 1)
```

```
# Linear prediction line
```

```
plt.plot(X_range, lin_reg.predict(X_range), color="red", label="Linear Regression")
```

```
# Polynomial prediction curve
```

```
X_range_poly = poly_features.transform(X_range)
```

```
plt.plot(X_range, poly_reg.predict(X_range_poly), color="green", label=f"Polynomial  
Regression (deg={degree})")
```

```
plt.xlabel("X")
```

```
plt.ylabel("y")
```

```
plt.title("Linear vs Polynomial Regression")
```

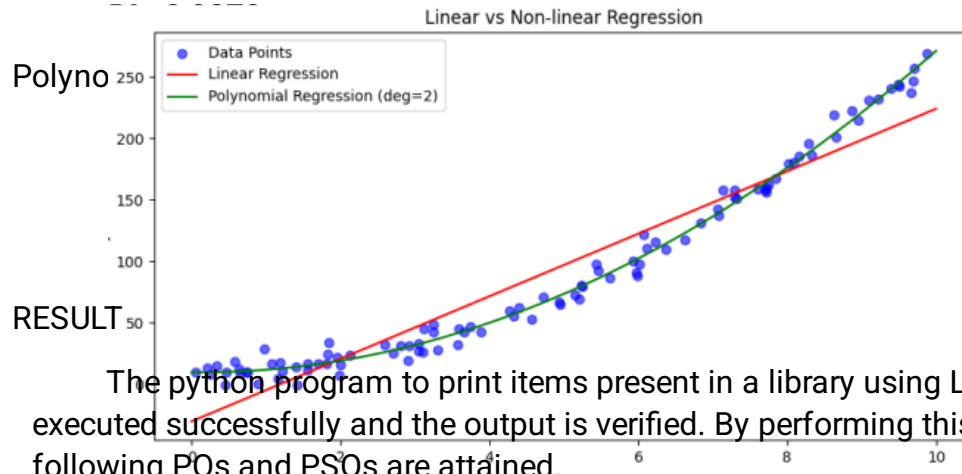
```
plt.legend()
```

```
plt.show()
```

OUTPUT

Linear Regression:

MSE: 442.24



POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PS0	PS0	PSO
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO 2	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2

Date:

AIM

To Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.

ALGORITHM

1. Load the dataset (Iris) and split it into features (X) and target (y).
2. Divide the dataset into training and testing sets.
3. Build a Decision Tree Classifier and train it using the training set.
4. Predict and evaluate the model on the test set (accuracy & classification report).
5. Classify a new sample and display the predicted class.

PROGRAM

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, accuracy_score
import matplotlib.pyplot as plt

# 1. Load real-world dataset (Iris)
iris = load_iris()
X = iris.data # Features: sepal length, sepal width, petal length, petal width
y = iris.target # Target: 0 = Setosa, 1 = Versicolor, 2 = Virginica

# Convert to DataFrame for clarity
df = pd.DataFrame(X, columns=iris.feature_names)
df["species"] = iris.target_names[y]

print("First 5 rows of dataset:")
print(df.head())

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 3. Train Decision Tree Classifier
clf = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)
clf.fit(X_train, y_train)

# 4. Predictions
y_pred = clf.predict(X_test)

print("\nModel Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=iris.target_names))
```

```
# 5. Visualize Decision Tree
```

```
plt.figure(figsize=(12, 6))
```

```
plot_tree(clf, filled=True, feature_names=iris.feature_names,
```

```
class_names=iris.target_names)
```

```
plt.show()
```

```
# 6. Classify a new sample
```

```
# Example: Sepal length=5.1, Sepal width=3.5, Petal length=1.4, Petal width=0.2
```

```
new_sample = [[5.1, 3.5, 1.4, 0.2]]
```

```
prediction = clf.predict(new_sample)
```

```
print("\nNew Sample:", new_sample)
```

```
print("Predicted Species:", iris.target_names[prediction][0])
```

OUTPUT

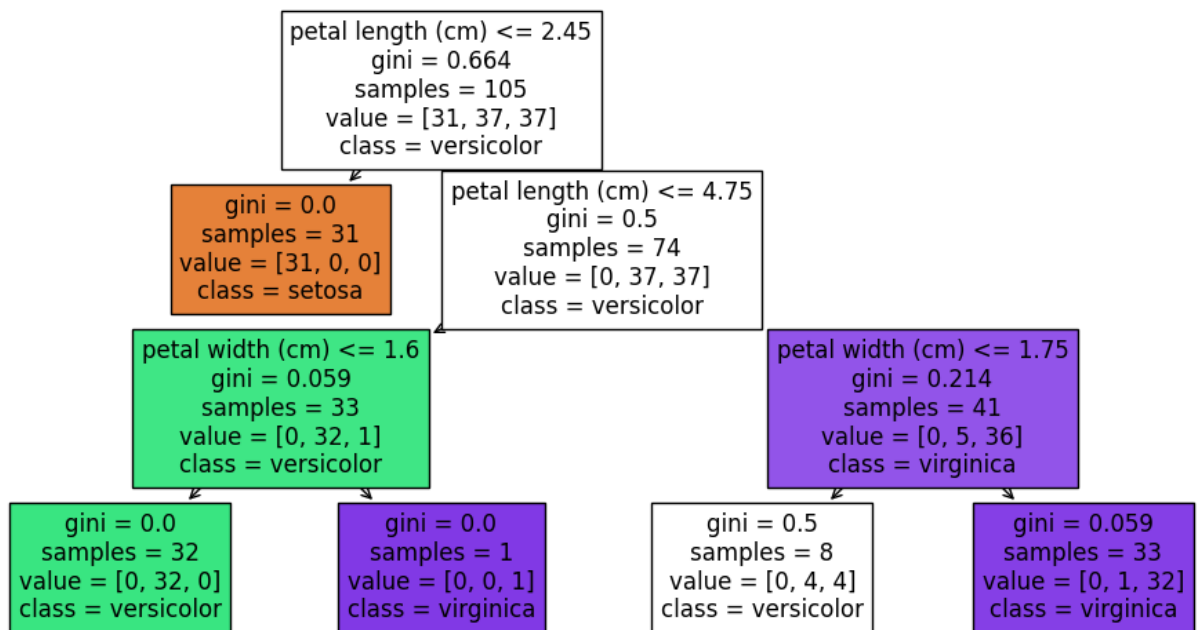
First 5 rows of dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Model Accuracy: 1.0

Classification Report:

	Precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13
accuracy		1.00	45	
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45



RESULT

Thus, the Implementation the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample was successfully executed and verified. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PS0	PS0	PSO
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO 2	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2

Date:

AIM

To Detect Spam mails using Support vector machine.

ALGORITHM

1. Load the dataset (text messages and labels).
2. Preprocess the text (clean, tokenize, and convert to numerical features using TF-IDF).
3. Split dataset into training and testing sets.
4. Train an SVM classifier on the training set.
5. Evaluate the model on the test set and test it with new sample emails.

PROGRAM

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
file_path = "spam.csv" # use your correct path
data = pd.read_csv(file_path, encoding="latin-1")

print("Sample Data:\n", data[["label", "text"]].head())

# Features (email text) and Labels (numeric spam/ham)
X = data["text"]
y = data["label_num"]

# Convert text to TF-IDF
vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)
X_features = vectorizer.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_features, y, test_size=0.3, random_state=42
```

```

)

# Train SVM Classifier
svm_clf = SVC(kernel="linear")
svm_clf.fit(X_train, y_train)

# Predictions
y_pred = svm_clf.predict(X_test)

print("\nModel Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=["Ham", "Spam"]))

# Test with new emails
new_emails = [
    "Congratulations! You've won a $1000 Walmart gift card. Click here to claim now.",
    "Hi John, just wanted to confirm our meeting tomorrow at 10 AM."
]

new_features = vectorizer.transform(new_emails)
predictions = svm_clf.predict(new_features)

print("\nNew Email Predictions:")
for email, label in zip(new_emails, predictions):
    print(f"'{email}'    {'Spam' if label == 1 else 'Ham'}")

```

OUTPUT

Sample Data:

	label	text
0	ham	Subject: enron methanol ; meter # : 988291\r\n...
1	ham	Subject: hpl nom for january 9 , 2001\r\n(see...
2	ham	Subject: neon retreat\r\nho ho ho , we ' re ar...
3	spam	Subject: photoshop , windows , office . cheap ...
4	ham	Subject: re : indian springs\r\nthis deal is t...

Model Accuracy: 0.9916237113402062

Classification Report:

precision recall f1-score support

Ham	1.00	0.99	0.99	1121
Spam	0.98	0.99	0.99	431

accuracy			0.99	1552
macro avg	0.99	0.99	0.99	1552
weighted avg	0.99	0.99	0.99	1552

New Email Predictions:

'Congratulations! You've won a \$1000 Walmart gift card. Click here to claim now.'

Spam

'Hi John, just wanted to confirm our meeting tomorrow at 10 AM.' Ham

RESULT

Thus, the Detection of Spam mails using Support vector machine is successfully executed and the output is verified. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PS0	PS0	PS0
PSOs	1	2	3	4	5	6	7	8	9	10	PO 11	PO 12	1	2	3

C03	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2
-----	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Date:

AIM

To Implement k-nearest neighbours classification.

ALGORITHM

- Step 1: Importing Libraries
- Step 2: Importing Dataset
- Step 3: Split Dataset
- Step 4: Training Model
- Step 5: Running Predictions
- Step 6: Checking Validation

PROGRAM

```
#Importing Libraries
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd
import numpy as np
from sklearn import datasets
import matplotlib.pyplot as plt
import seaborn as sn

#Importing Dataset
iris = datasets.load_iris()
iris_data = iris.data
iris_labels = iris.target
print(iris_data)

#Split Dataset
x_train, x_test, y_train, y_test = train_test_split(iris_data, iris_labels, test_size=0.20)

#Training Model
classifier = KNeighborsClassifier(n_neighbors=6)
classifier.fit(x_train, y_train)
y_pred = classifier.predict(x_test)

#Running Predictions
print("Classification Report:\n")
print(classification_report(y_test, y_pred))

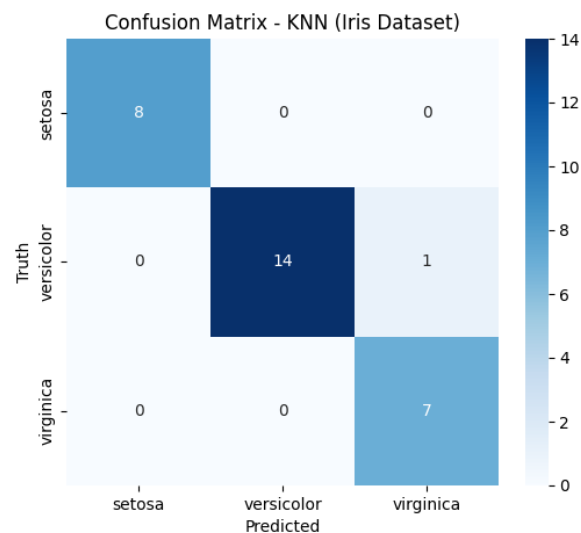
#Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
```

```
#Checking Validation (Heatmap)
plt.figure(figsize=(6,5))
sn.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=iris.target_names,
yticklabels=iris.target_names)
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.title("Confusion Matrix - KNN (Iris Dataset)")
plt.show()
```

OUTPUT

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8
1	1.00	0.93	0.97	15
2	0.88	1.00	0.93	7
accuracy			0.97	30
macro avg	0.96	0.98	0.97	30
weighted avg	0.97	0.97	0.97	30



RESULT

Thus, the Implementation of k-nearest neighbours classification is executed successfully and the output is verified. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PS0	PS0	PS0
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO3	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2

Date:

AIM

To Implement k-means clustering algorithm..

PROCEDURE

1. Load dataset – Iris dataset with 150 samples and 4 features.
2. Standardize features – ensures all features have equal weight.
3. KMeans(n_clusters=3) – since Iris has 3 species.
4. Fit & Predict – assigns each sample to a cluster.
5. Visualize – shows clusters and centroids.

PROGRAM

Importing Libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn import datasets

from sklearn.preprocessing import StandardScaler

Load Iris dataset

iris = datasets.load_iris()

X = iris.data # Features

y = iris.target # True labels (not used in clustering)

Standardize the data (important for clustering)

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

Apply K-Means Clustering

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

kmeans.fit(X_scaled)

Predicted clusters

y_kmeans = kmeans.predict(X_scaled)

Print Cluster Centers

print("Cluster Centers:\n", kmeans.cluster_centers_)

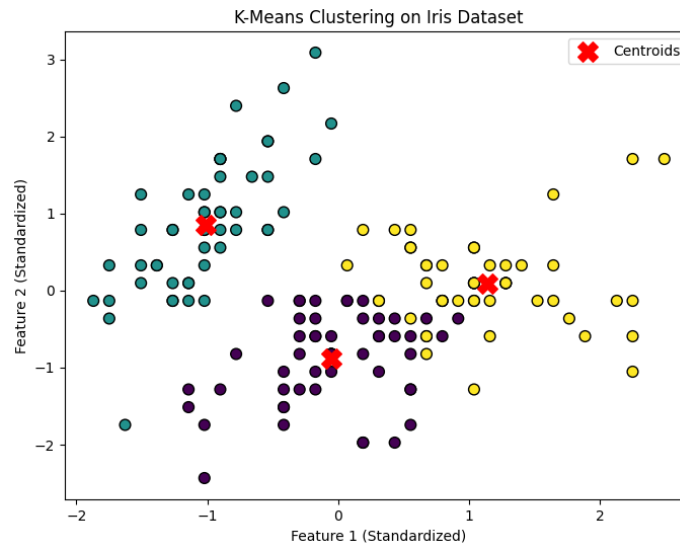
Visualization (using first two features for 2D plot)

plt.figure(figsize=(8,6))

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_kmeans, cmap="viridis", marker="o",
edgecolor="k", s=60)

```
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
            c="red", marker="X", s=200, label="Centroids")
plt.xlabel("Feature 1 (Standardized)")
plt.ylabel("Feature 2 (Standardized)")
plt.title("K-Means Clustering on Iris Dataset")
plt.legend()
plt.show()
```

OUTPUT



RESULT

Thus, the Implementation of k-means clustering algorithm is executed successfully. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PS0	PS0	PSO
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO4	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2

Ex No: 8

IMPLEMENT SENTIMENT ANALYSIS USING RANDOM FOREST
OPTIMIZATION ALGORITHM.

Date:

AIM

To Implement sentiment analysis using random forest optimization algorithm.

ALGORITHM

1. Load IMDB dataset (50,000 reviews).
2. Convert integer sequences back to words.
3. Apply TF-IDF vectorization to extract features.
4. Train a Random Forest classifier.
5. Evaluate with accuracy + classification report.
6. Test with new custom reviews.

PROGRAM

Sentiment Analysis with Random Forest

Import Libraries

```
import numpy as np
import pandas as pd
from tensorflow.keras.datasets import imdb
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

1. Load IMDB dataset (pre-tokenized as integers)

```
num_words = 5000 # Only keep top 5000 words
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=num_words)
```

2. Decode integers back to words for TF-IDF processing

```
word_index = imdb.get_word_index()
reverse_index = {value: key for (key, value) in word_index.items()}
```

def decode_review(encoded_review):

```
    return " ".join([reverse_index.get(i - 3, "?") for i in encoded_review if i >= 3])
```

```
X_train_text = [" ".join([reverse_index.get(i - 3, "?") for i in seq if i >= 3]) for seq in X_train]
```

```
X_test_text = [" ".join([reverse_index.get(i - 3, "?") for i in seq if i >= 3]) for seq in X_test]
```

3. Convert text to TF-IDF features

```
vectorizer = TfidfVectorizer(max_features=5000, stop_words="english")
X_train_features = vectorizer.fit_transform(X_train_text)
X_test_features = vectorizer.transform(X_test_text)
```

4. Train Random Forest Classifier

```
rf_clf = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
rf_clf.fit(X_train_features, y_train)
```

5. Predictions & Evaluation

```
y_pred = rf_clf.predict(X_test_features)
```

```
print("\nModel Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=["Negative", "Positive"]))
```

6. Test with custom reviews

```
new_reviews = [
    "This movie was absolutely fantastic! The story, the acting, everything was great.",
    "I hated this movie. It was boring and a complete waste of time."
]
```

```
new_features = vectorizer.transform(new_reviews)
predictions = rf_clf.predict(new_features)
```

```
print("\nNew Review Predictions:")
for review, label in zip(new_reviews, predictions):
    print(f'"{review}"    {'Positive' if label == 1 else 'Negative'})
```

OUTPUT

Model Accuracy: 0.8472

Classification Report:

	precision	recall	f1-score	support
Negative	0.84	0.86	0.85	12500
Positive	0.86	0.83	0.84	12500
accuracy			0.85	25000
macro avg	0.85	0.85	0.85	25000
weighted avg	0.85	0.85	0.85	25000

New Review Predictions:

'This movie was absolutely fantastic! The story, the acting, everything was great.'

Positive

'I hated this movie. It was boring and a complete waste of time.' Negative

RESULT

Thus, Implementation of sentiment analysis using random forest optimization algorithm is executed successfully. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PSO	PSO	PSO
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO4	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2

Date:

AIM

To Write a program to construct a Bayesian network considering medical data

ALGORITHM

1. Step 1: Import the required libraries
2. Step 2: Load the heart dataset
3. Step 3: Explore the dataset
4. Step 4: Build the Model

PROGRAM

```
#Import the required libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
#Import the dataset
```

```
df=pd.read_csv('heart.csv')
```

```
#Build the Model
```

```
from pgmpy.models import BayesianModel
```

```
from pgmpy.estimators import MaximumLikelihoodEstimator,BayesianEstimator
```

```
model=BayesianModel([('age','target'),('sex','target'),('exang','target'),('cp','target'),  
('target','restecg'),('target','chol')])
```

```
model.fit(df,estimator=MaximumLikelihoodEstimator)
```

```
print(model.get_cpds('age'))
```

```
OUTPUT
```

```
+-----+-----+
```

```
| age(29) | 0.00330033 |
```

```
+-----+-----+
```

```
| age(34) | 0.00660066 |
```

```
+-----+-----+
```

```
| age(35) | 0.0132013 |
```

```
+-----+-----+
```

```
| age(37) | 0.00660066 |
```

```
+-----+-----+
```

```
| age(38) | 0.00990099 |
```

```
+-----+-----+
```

```
| age(39) | 0.0132013 |
```

```
+-----+-----+
```

```
print(model.get_cpds('sex'))
```

```
OUTPUT
```

```
| sex(0) | 0.316832 |
```

```
+-----+-----+
| sex(1) | 0.683168 |
```

```
print(model.get_cpds('exang'))
```

```
OUTPUT
```

```
| exang(0) | 0.673267 |
```

```
+-----+-----+
```

```
| exang(1) | 0.326733 |
```

```
+-----+-----+
```

```
print(model.get_cpds('chol'))
```

```
OUTPUT
```

```
|target |target(0)      |target(1)                |
+-----+-----+-----+
|chol(126)| 0.0            | 0.006060606060606061    |
+-----+-----+-----+
|chol(131)| 0.007246376811594203 | 0.0                      |
+-----+-----+-----+
|chol(141)| 0.0            | 0.006060606060606061    |
+-----+-----+-----+
|chol(149)| 0.007246376811594203 | 0.006060606060606061    |
+-----+-----+-----+
|chol(157)| 0.0            | 0.006060606060606061    |
+-----+-----+-----+
```

RESULT

Thus, the python program to construct a Bayesian network considering medical data is executed successfully. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PS0	PS0	PS0
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
C05	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2

Date:

AIM

To Implement facial recognition application with artificial neural network.

ALGORITHM

- Step 1: Import the data
- Step 2: Transform the data
- Step 3: Construct the tensor
- Step 4: Build the model
- Step 5: Train and evaluate the model
- Step 6: Improve the model

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import mglearn
from sklearn.datasets import fetch_lfw_people
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf

# Load dataset
people = fetch_lfw_people(min_faces_per_person=20, resize=0.7)
print("Data shape:", people.data.shape)
print("Target shape:", people.target.shape)

image_shape = people.images[0].shape

# Plot sample faces
fig, axes = plt.subplots(2, 5, figsize=(15, 8), subplot_kw={'xticks': (), 'yticks': ()})
for target, image, ax in zip(people.target, people.images, axes.ravel()):
    ax.imshow(image)
    ax.set_title(people.target_names[target])

# Limit dataset to 50 images per person
mask = np.zeros(people.target.shape, dtype=bool)
for target in np.unique(people.target):
    mask[np.where(people.target == target)[0][:50]] = 1

X_people = people.data[mask]
y_people = people.target[mask]
```

```

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_people, y_people, stratify=y_people, random_state=0
)

# KNN without PCA
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_train, y_train)
print("KNN Test accuracy (raw features): {:.2f}".format(knn.score(X_test, y_test)))

# PCA
pca = PCA(n_components=100, whiten=True, random_state=0).fit(X_train)
X_train_pca = pca.transform(X_train)
X_test_pca = pca.transform(X_test)
print("X_train_pca.shape:", X_train_pca.shape)

# KNN with PCA
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_train_pca, y_train)
print("KNN Test accuracy (PCA features): {:.2f}".format(knn.score(X_test_pca, y_test)))

# Plot PCA components
fig, axes = plt.subplots(3, 5, figsize=(15, 12), subplot_kw={'xticks': (), 'yticks': ()})
for i, (component, ax) in enumerate(zip(pca.components_, axes.ravel())):
    ax.imshow(component.reshape(image_shape), cmap="viridis")
    ax.set_title("{} component".format(i + 1))

# Rescale features
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train.astype(int))
X_test_scaled = scaler.transform(X_test.astype(int))

# TensorFlow DNN Classifier (Estimator API)
feature_columns = [tf.feature_column.numeric_column('x',
shape=X_train_scaled.shape[1:])]

estimator = tf.compat.v1.estimator.DNNClassifier(
    feature_columns=feature_columns,
    hidden_units=[300, 100],
    n_classes=len(np.unique(y_train)),
    model_dir='./dnn_model'
)

train_input = tf.compat.v1.estimator.inputs.numpy_input_fn(
    x={"x": X_train_scaled},

```

```

y=y_train,
batch_size=50,
shuffle=True,
num_epochs=None
)
estimator.train(input_fn=train_input, steps=1000)
eval_input = tf.compat.v1.estimator.inputs.numpy_input_fn(
    x={"x": X_test_scaled},
    y=y_test,
    shuffle=False,
    num_epochs=1
)

```

```

results = estimator.evaluate(eval_input)
print("DNN Test Accuracy:", results['accuracy'])

```

OUTPUT

```

Data shape: (3023, 5655)
Target shape: (3023,)
KNN Test accuracy (raw features): 0.23
X_train_pca.shape: (1387, 100)
KNN Test accuracy (PCA features): 0.29
DNN Test Accuracy: 0.40

```

RESULT

Thus, Implementation of facial recognition application with artificial neural network is executed successfully. By performing this experiment the following POs and PSOs are attained.

POs	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PO	PSO	PSO	PSO
PSOs	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO5	3	2	1	2	1	-	-	2	2	1	-	1	3	2	2